# SolutionArchitectAI. Ключевые технические решения

Технические решения

Цель документа — зафиксировать планируемые технические решения по реализации MVP SolutionArchitectAI для согласования с системными архитекторами, техническими руководителями, Python- и frontend-разработчиками.

## Содержание


**1. Назначение и общая архитектура** 2

**2. Backend** 3

**3. Хранение данных** 3

**4. Искусственный интеллект** 4

**5. Пользовательский интерфейс** 5

**6. Контроль качества кода и линтеры** 5


---

## 1. Назначение и общая архитектура

SolutionArchitectAI помогает перейти от требований и корпоративного контекста к согласованной архитектуре. Приложение объединяет архитектурную модель, диаграммы, документы, чат с ИИ, проверки и историю решений. Предложения ИИ рассматривает и принимает архитектор.

Frontend на React и TypeScript предоставляет пользовательский интерфейс. Backend на Python и FastAPI выполняет предметные операции, агентный процесс и работу с хранилищами.

### Развёртывание

Для локального запуска планируются две конфигурации Docker Compose: основная `docker-compose.yml` с приложением, PostgreSQL и локальным PlantUML Server; дополнительная `docker-compose.ln.yml` с vLLM и локальной моделью. Такой запуск не требует внешнего LLM API-ключа. Данные Git и загруженные файлы сохраняются между запусками; MinIO можно включить для проверки S3-совместимого хранения.

Для production планируется Red Hat OpenShift. Git-репозитории моделей размещаются в сохраняемом хранилище; в первоначальной конфигурации запись в Git выполняет один активный экземпляр backend. Сохранность PostgreSQL и файлового хранилища обеспечивается отдельно.

## 2. Backend

Планируемый стек:

- **Python 3.13** — язык и целевая версия серверной части.
- **uv** — управление зависимостями, окружениями и lock-файлом для воспроизводимой установки.
- **FastAPI** — HTTP API приложения.
- **Dishka** — внедрение зависимостей и управление их жизненным циклом.
- **Pydantic** — проверка и преобразование структурированных данных.
- **Pydantic Settings** — загрузка и проверка конфигурации.
- **SQLAlchemy** — взаимодействие с PostgreSQL.
- **Alembic** — миграции структуры базы данных.

Единая предметная модель архитектуры реализуется на Python. Она описывает системы и компоненты, данные и интерфейсы, отношения между объектами, архитектурные решения, а также правила и ограничения изменений. Каждый объект имеет устойчивый идентификатор и может использоваться в нескольких представлениях. Удаление графического отображения из одной диаграммы не удаляет объект из общей модели. JSON сериализует модель для сохранения её редакций в Git; предметные правила остаются в Python.

## 3. Хранение данных

- **PostgreSQL** — пользователи, проекты, задачи, чаты, сообщения, состояние агентных процессов, согласования и журнал операций. Здесь же сохраняются контрольные точки LangGraph.
- **Git** — редакции общей архитектурной модели и её JSON-представления, история изменений, архитектурные решения и представления.
- **Файловое хранилище** — загруженные документы, вложения и крупные файлы. Планируются локальная файловая система и S3-совместимый вариант; для локальной проверки S3-интеграции возможен отдельный необязательный контейнер MinIO.

PostgreSQL поддерживает текущее состояние совместной работы, Git — версионность архитектуры, файловое хранилище — крупные материалы. Поэтому они выполняют разные задачи.

## 4. Искусственный интеллект

**LangChain** через `langchain-core` и `langchain-openai` подключает настраиваемый OpenAI-совместимый сервер модели. Pydantic проверяет структуру ответов; предметные проверки и решение человека остаются обязательными. Возможности структурированного вывода проверяются для конкретной пары модели и сервера.

**LangGraph** управляет агентным процессом с сохранением состояния, остановкой для ответа человека и возобновлением. PostgreSQL checkpointer сохраняет контрольные точки процесса.

### Роли агентов

- **Агент извлечения требований** выявляет требования, противоречия и пробелы со ссылками на источники.
- **Агент уточнений** формирует вопросы архитектору и учитывает ответы.
- **Агент проектирования** предлагает объекты, связи, решения и представления.
- **Агент архитектурного ревью** независимо проверяет предложенную архитектуру.

Агенты не согласовывают собственные предложения и не выполняют merge.

### Локальная модель

Дополнительная конфигурация Docker Compose предусматривает vLLM с локальной моделью. Для компьютера с подходящим GPU рассматривается `openai/gpt-oss-20b`; требования к оборудованию и поддержка нужных возможностей проверяются при реализации. Для менее мощного компьютера предусматривается более компактная совместимая модель либо демонстрационный режим с fake LLM. Последний показывает работу интерфейса и процессов, но не качество агентного проектирования.

## 5. Пользовательский интерфейс

**React**, **TypeScript** и **Vite** образуют веб-интерфейс. **React Router** отвечает за адресуемые разделы, **TanStack Query** — за загрузку и обновление серверных данных. **shadcn/ui** и **Tailwind CSS** используются для компонентов и оформления; **assistant-ui** — для чата с ИИ через собственный адаптер FastAPI.

**React Flow** обеспечивает интерактивное редактирование диаграмм: выбор и перемещение элементов, работу со связями и масштабирование. Предметные операции и допустимость изменений определяет backend, а расположение элементов сохраняется отдельно для каждого представления.

**PlantUML** генерирует публикуемые схемы из проверенной модели через локальный сервер. Для возможного будущего обмена с Archi предусмотрен отдельный экспорт предметной модели в **ArchiMate Model Exchange XML**.

## 6. Контроль качества кода и линтеры

### Backend

- **Ruff** — основной линтер и единственный форматтер Python.
- **Mypy** — строгая статическая проверка типов значимых частей backend.
- **Pylint** — дополнительный анализ качества и межмодульных ошибок.
- **Bandit** — поиск потенциальных проблем безопасности.
- **import-linter** — проверка разрешённых зависимостей между пакетами.
- **wemake-python-styleguide** — точечная проверка когнитивной сложности WPS231.
- **pytest** — модульные и интеграционные тесты.

Пересекающиеся диагностики Ruff, Pylint и Bandit будут разделены в конфигурации проверок.

### Frontend

- **ESLint** — линтинг TypeScript и React.
- **Prettier** — форматирование кода.
- **TypeScript strict** — строгая проверка типов.
- **Vitest** — модульные и компонентные тесты.
- **Playwright** — сквозные проверки приложения в браузере.